

API Reliability & Incident Management Platform

Engineering PRD, System Design & Execution Plan — v2.0

Codename: pulse | **Tagline:** "Mini Datadog + PagerDuty for small teams"

This document is written to be consumed by an AI coding agent (Cursor / Claude Code / Antigravity) as the single source of truth for implementation. Every section maps to concrete files, commands, and acceptance criteria — not just prose. Treat each numbered task as a discrete unit of work with its own commit and PR.

0. How to use this document (for the AI agent)

1. Work top-to-bottom through **Section 9 (Execution Plan)**. Do not skip ahead to later milestones.
2. Before writing code for a milestone, re-read the relevant subsection of **Section 6 (Architecture)** and **Section 7 (Data & API Contracts)** — they are the contract; do not invent schemas ad hoc.
3. Every milestone ends with: tests passing → lint passing → CI green on a feature branch → PR opened → merged to `main`. No milestone is "done" until CI is green on `main`.
4. If a requirement is ambiguous, prefer the interpretation that keeps multi-tenant isolation and durability guarantees intact — these are the two non-negotiable properties of this system.

5. Never commit secrets. Use `.env.example` as the contract for required environment variables; real values live in the deploy platform's secret store and in GitHub Actions/Jenkins credentials, never in git.
-

1. Problem Statement

Small dev teams and indie hackers can't justify Datadog or PagerDuty pricing to answer one question: *"Is my API up right now, and did the right person get paged when it wasn't?"* Today they find out from an angry user on Twitter/X instead of their own tooling.

Product: register an HTTP(S) endpoint → get it health-checked on a schedule → automatically open an incident with a root-cause hypothesis when it breaks → track the incident to resolution → surface uptime history on a public status page.

2. Goals, Non-Goals, Success Metrics

2.1 Goals (v1)

- Register and monitor HTTP(S) endpoints on a schedule with durable, crash-safe scheduling.
- Detect downtime/degradation without false positives from single blips ("flapping" protection).
- Auto-create incidents, notify the right people, track state to resolution.
- Provide an AI-assisted, **grounded** first hypothesis for root cause (no hallucinated causes).
- Ship an uptime dashboard and a public, unauthenticated status page.

2.2 Non-Goals (v1)

- Full APM / distributed tracing.
- On-call scheduling complexity (rotations, multi-tier escalation policies).
- Multi-region check origins.
- Native mobile app.
- SSO/SAML (email+password + JWT is sufficient for v1).

2.3 Success Metrics (how we'll know v1 is "good")

Metric	Target
Incident detection latency	< 1 check interval + retry window (e.g., ≤ 2 min for a 1-min interval)
False-positive incident rate	< 5% of opened incidents auto-resolve within 1 check cycle
Dashboard p95 load time	< 500 ms with 30 days of check history per API
CI pipeline duration	< 8 min from push to green
Test coverage (incident state machine + auth)	≥ 85% line coverage on those modules

3. Personas & Core Use Cases

Persona	Need
Indie dev	Zero-friction setup, free tier, email/Slack alert
Small team lead (3–8 ppl)	Team dashboard, shared incidents, uptime SLA tracking
Freelancer	Shareable public status page per client
On-call engineer	Incident timeline, historical failure patterns, AI hypothesis
Engineering manager	MTTR trend, monthly reliability review export
New team member	Incident timeline as tribal-knowledge documentation

4. Functional Requirements (with acceptance criteria)

FR-1 Auth & Teams

- Email/password signup, JWT-based session, password hashing via `bcrypt` (cost factor 12).
- A user creates or joins exactly one team via invite link/token (v1: single-team membership).
- Roles: `admin` (manage APIs, invite/remove members, alert rules), `member` (view + comment on incidents).
- **AC:** A `member` calling an admin-only endpoint receives `403` with a machine-readable error code, not a silent no-op.

FR-2 API Registration

- Fields: `name` , `url` , `method` (GET/POST/HEAD), `expectedStatus` , `latencyThresholdMs` , `intervalSeconds` (min 30s for v1 to bound cost), optional `headers` , optional `authToken` (encrypted at rest), optional `body` .
- **AC:** URL must be validated as a well-formed, public-reachable HTTP(S) URL at creation time (reject `localhost` /private IP ranges to prevent SSRF against internal infra).

FR-3 Health Checks

- Scheduled ping per API on its configured interval; each check records `status` , `latencyMs` , `statusCode` , `checkedAt` , `errorMessage?` .
- **AC:** A worker crash/restart mid-check must not duplicate or silently drop a scheduled check (Temporal guarantee – see §6.3).

FR-4 Incident Engine (state machine)

- `N` consecutive failures (default `N=3`) → open incident (status `OPEN`).
- `M` consecutive successes (default `M=2`) while `OPEN` / `INVESTIGATING` → auto-resolve (status `RESOLVED`).
- **AC:** A single isolated failure followed by a success must never create an incident (unit-tested explicitly – this is the "flapping" guard).
- **AC:** Incident transitions are append-only in `timeline[]` – no destructive overwrites.

FR-5 Notifications

- Email required for v1 (incident opened + resolved). Slack webhook stretch goal. In-app notification always fires.

- **AC:** If the email provider call fails, the incident is still created/updated — notification is fire-and-forget, never blocking, and failures are logged + retried via a queue, not swallowed silently.

FR-6 Incident Timeline & Comments

- Chronological, append-only log: state changes, linked failed `CheckResult` s, free-text comments from team members.

FR-7 AI Root-Cause Analysis

- On incident creation, a LangGraph agent pulls the last K `CheckResult` failures (+ status codes + timing) for that API, and produces `aiSummary` + `aiSuggestedCause` + suggested next debug step.
- **AC:** The prompt sent to Claude must include only real, retrieved `CheckResult` data — never free-form speculation without grounding context attached. If context retrieval returns zero rows, skip the AI call and mark `aiSummary:` `null` rather than inventing a cause.
- **AC:** If the Claude API call errors or times out (>10s), the incident is still created; `aiSummary` is set to `null` and a retry job is queued.

FR-8 Dashboards

- Per-API uptime % (24h/7d/30d), latency graph, team-wide incident feed.
- **AC:** Uptime % must be computed from pre-aggregated rollups, not a full scan of raw `CheckResult` rows on every page load (see §6.4 caching strategy).

FR-9 Public Status Page

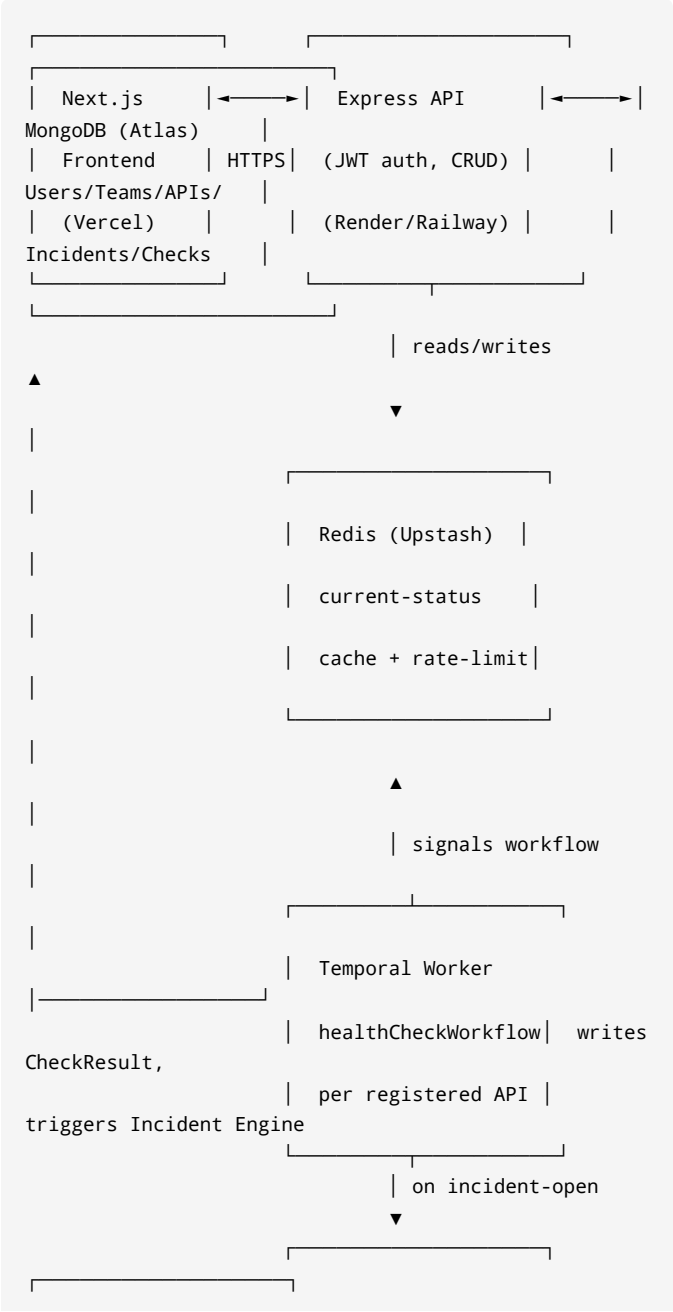
- Read-only, unauthenticated, per-team-or-per-API. Shows current status + historical uptime only — **must not** leak `authToken` , internal `teamId` relationships beyond what's displayed, or any other team's data.
 - **AC:** A negative security test exists proving the public endpoint cannot be parameterized to enumerate/leak other teams' APIs.
-

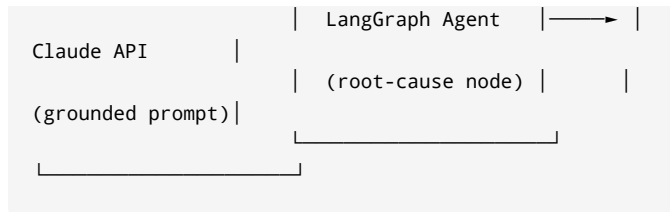
5. Non-Functional Requirements

- **Durability:** scheduling survives app-server restarts and deploys (Temporal, not `setInterval` / `node-cron`).
 - **Latency:** incident detection < 1 check interval + retry window.
 - **Scalability of reads:** dashboard queries stay fast as check history grows (Redis cache for "current status"; MongoDB TTL index on raw `CheckResult`).
 - **Multi-tenancy:** every query that touches `Api` , `Incident` , or `CheckResult` must be scoped by `teamId` . This is enforced at the data-access-layer, not ad hoc per route (see §6.5).
 - **Security:** JWT auth, bcrypt password hashing, encrypted-at-rest third-party auth tokens, SSRF-safe URL validation, rate limiting on public endpoints.
 - **Observability:** structured JSON logs, request IDs, health/readiness endpoints for the API and the Temporal worker.
-

6. System Architecture

6.1 High-level component diagram (textual)





6.2 Repository layout (monorepo, npm workspaces)

```

pulse/
├── .github/workflows/          # CI/CD pipelines
(see §10)
├── apps/
│   ├── frontend/              # Next.js + TS +
Tailwind
│   └── backend/                # Express API +
Temporal worker + LangGraph agent
│   ├── src/
│   │   ├── config/            # env loading,
constants
│   │   └── db/                # mongoose models +
connection
│   │   ├── modules/
│   │   │   ├── auth/
│   │   │   ├── teams/
│   │   │   ├── apis/
│   │   │   ├── checks/
│   │   │   ├── incidents/
│   │   │   ├── notifications/
│   │   │   └── ai-analysis/
│   │   ├── temporal/
│   │   │   ├── workflows/
│   │   │   ├── activities/
│   │   │   └── worker.ts
│   │   ├── middleware/        # auth guard, team-
scope guard, rate limit
│   │   └── lib/                # redis client,
logger, error classes
│   │   └── server.ts
│   └── test/                  # unit +

```

```

integration tests
├── package.json
├── packages/
│   └── shared-types/          # Zod schemas + TS
types shared FE/BE
├── docker-compose.yml        # local Mongo +
Redis + Temporal dev stack
├── .env.example
├── package.json              # workspaces root
└── README.md

```

6.3 Why Temporal (not `node-cron` / `setInterval`)

A naive in-process scheduler loses all schedule state on deploy/restart/crash — silent monitoring gaps are the one failure mode this product cannot tolerate (it's a monitoring product; if it silently stops monitoring, it has failed at its core job). Temporal workflows are durable: workflow state is persisted server-side, and on worker restart, the workflow resumes exactly where it left off. Each registered `Api` gets one long-running `healthCheckWorkflow` that sleeps between checks using `sleep()` inside the workflow (not a cron poll), calls the `pingApi` activity, and Temporal's activity retry policy handles transient network blips before they ever reach the incident engine.

6.4 Caching strategy

- Redis key `status:{apiId}` → `{status, lastCheckedAt, latencyMs}` , written on every check, read on every dashboard load. TTL not required (always overwritten), but set a generous TTL (e.g. 1h) as a self-healing safety net against orphaned keys.
- Uptime % for 24h/7d/30d windows is computed by a scheduled aggregation (Temporal cron workflow, hourly) into an `UptimeRollup` collection, not computed live from raw `CheckResult` on each dashboard request.

- `CheckResult` has a MongoDB TTL index (30-day retention) to bound collection growth.

6.5 Multi-tenant isolation strategy

All data-access goes through a repository layer

(`src/modules/*/repository.ts`) whose functions require

`teamId` as a mandatory first argument — there is no code path to query `Api` / `Incident` / `CheckResult` without it. A lint rule / code-review checklist item enforces this: **no raw Mongoose model calls outside a repository file.**

6.6 AI grounding strategy (LangGraph)

Graph nodes: `fetchRecentFailures` (Mongo query, `teamId+apild` scoped) → `buildContext` (format last K failures + status-code histogram + deploy-correlation if available) → `callClaude` (structured prompt: "Given ONLY the following observed data, suggest the most likely cause and one debugging step. If the data is insufficient, say so explicitly.") → `parseAndPersist` (attach `aiSummary` / `aiSuggestedCause` to the `Incident`). If `fetchRecentFailures` returns empty, the graph short-circuits before calling Claude.

7. Data Models & API Contracts

7.1 MongoDB schemas (Mongoose, abbreviated)

```
// User
{ _id, name, email: { type: String, unique: true },
  passwordHash, teamId, role: 'admin'|'member',
  createdAt }

// Team
```

```

{ _id, name, members: [{ userId, role }], createdAt }

// Api
{ _id, teamId, name, url, method, expectedStatus,
  latencyThresholdMs,
  intervalSeconds, headers: Map, authTokenEncrypted,
  isActive, isPublic, createdAt }

// CheckResult -- TTL index on checkedAt, expires:
30d
{ _id, apiId, teamId, status: 'UP'|'DOWN'|'DEGRADED',
  latencyMs, statusCode,
  errorMessage, checkedAt }

// Incident
{ _id, apiId, teamId, startedAt, resolvedAt,
  status: 'OPEN'|'INVESTIGATING'|'RESOLVED',
  aiSummary, aiSuggestedCause,
  timeline: [{ type, message, actorId?, at }] }

// UptimeRollup -- written hourly by scheduled
workflow
{ _id, apiId, teamId, windowStart, windowEnd,
  uptimePct, avgLatencyMs }

```

7.2 REST API surface (v1)

```

POST  /api/v1/auth/signup
POST  /api/v1/auth/login
POST  /api/v1/teams/:teamId/invite

GET    /api/v1/apis                               (team-
scoped, from JWT)
POST   /api/v1/apis
GET    /api/v1/apis/:id
PATCH /api/v1/apis/:id
DELETE /api/v1/apis/:id

GET    /api/v1/apis/:id/checks?window=24h|7d|30d
GET    /api/v1/apis/:id/uptime?window=24h|7d|30d

```

```

GET    /api/v1/incidents?status=OPEN|RESOLVED
GET    /api/v1/incidents/:id
POST   /api/v1/incidents/:id/comments
PATCH /api/v1/incidents/:id          (status
transitions only, admin)

GET    /api/v1/public/status/:teamSlug/:apiSlug
(no auth, rate-limited)

GET    /healthz                      (backend
liveness)
GET    /readyz                      (backend
readiness: mongo+redis ping)

```

Every non-public route requires `Authorization: Bearer <jwt>` and is scoped via `req.teamId` extracted from the verified token — never trust a `teamId` in the request body/query for a mutating call.

8. Tech Stack (with rationale, for interview defense)

Layer	Choice	Why
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind	SSR for the public status page (SEO + fast first paint), file-based routing
Backend	Node.js + Express + TypeScript	Explicit control over middleware chain for the team-scoping guard; simpler mental

Layer	Choice	Why
		model than mixing into Next API routes
DB	MongoDB Atlas	Schema flexibility for <code>Api</code> config (variable headers/body), TTL indexes native for check-log retention
Cache	Redis (Upstash)	Sub-ms current-status reads; rate limiting via <code>INCR + EXPIRE</code>
Durable scheduling	Temporal Cloud (or self-hosted)	Crash-safe long-running workflows per API — the centerpiece hard-engineering decision
AI orchestration	LangGraph + Claude API	Structured, inspectable multi-node agent rather than a single opaque prompt call
CI/CD	GitHub Actions (primary), Jenkinsfile (alt/on-prem)	See §10
Hosting	Render/Railway (backend + worker), Vercel (frontend), Atlas, Upstash	Managed, low-ops, cheap for a v1

9. Execution Plan — Milestone by Milestone

Each milestone = one feature branch, one PR, CI must be green before merge. Commit early and often within a milestone; do not batch a whole day into one commit.

M0 — Repo bootstrap

Branch: chore/repo-scaffold

- [] `npm init workspaces root`; create `apps/frontend`, `apps/backend`, `packages/shared-types`.
- [] `.editorconfig`, root `tsconfig.base.json`, ESLint + Prettier configs shared via root.
- [] `.env.example` listing every required var (`MONGO_URI`, `REDIS_URL`, `JWT_SECRET`, `TEMPORAL_ADDRESS`, `ANTHROPIC_API_KEY`, `RESEND_API_KEY`, `NODE_ENV`).
- [] `docker-compose.yml` : local Mongo, Redis, Temporal dev server (`temporalio/auto-setup`).
- [] README skeleton with architecture diagram (embed §6.1) and setup steps.
- [] `.github/workflows/ci.yml` stub (lint + typecheck job only, so CI is green from commit #1).
- **Commit checkpoints:** `chore: workspace scaffold`, `chore: shared eslint/prettier/tsconfig`, `chore: docker-compose dev stack`, `ci: lint+typecheck workflow`
- **DoD:** `npm install` && `npm run lint` && `npm run typecheck` succeeds locally and in CI on push.

M1 — Auth, Teams, multi-tenant guard

Branch: feat/auth-teams

- [] Mongoose models: `User`, `Team`.

- [] `POST /auth/signup` , `POST /auth/login` (bcrypt + JWT, 7-day expiry, refresh not required for v1).
- [] `teamScopeGuard` middleware: verifies JWT, attaches `req.teamId / req.userId / req.role` .
- [] `adminOnly` middleware.
- [] Team invite endpoint (token-based, single use).
- [] Unit tests: password hashing round-trip, JWT verify/expiry, guard rejects missing/invalid/expired token, `403` for member hitting admin route.
- **Interview Qs to be ready for:** JWT storage (httpOnly cookie vs. header — state your choice and why), team-isolation-at-query-layer argument, what happens if a route forgets the guard (answer: it can't compile/pass review — see repository-layer rule §6.5).
- **DoD:** `npm test -w apps/backend` green, ≥85% coverage on `auth` + `middleware` .

M2 — API Registration CRUD

Branch: `feat/api-registration`

- [] `Api` model + Zod validation schema in `packages/shared-types` (shared with frontend form validation).
- [] SSRF guard: reject private/loopback/link-local IP ranges and non-http(s) schemes at creation and update time.
- [] Full CRUD, all routes team-scoped via repository layer.
- [] Encrypt `authToken` at rest (AES-256-GCM, key from env, never logged).
- **DoD:** integration test proves Team B cannot read/update/delete Team A's `Api` even with a valid Team B JWT.

M3 — Temporal health-check engine

Branch: feat/temporal-health-checks

- [] Temporal worker process (`src/temporal/worker.ts`), separate `npm run worker` script/deploy target from the API server.
- [] `healthCheckWorkflow(apiId)` : infinite loop – `sleep(intervalSeconds)` → call `pingApi` activity → persist result → continue-as-new periodically to bound history size.
- [] `pingApi` activity: HTTP call with configurable timeout, retry policy (max 3 attempts, exponential backoff) for transient errors only (timeouts/5xx), not for a clean 4xx.
- [] On `Api` creation → start the workflow (`workflowId = healthcheck-{apiId}` , idempotent start). On `Api` deactivation/deletion → signal/terminate the workflow.
- [] Write `CheckResult` ; update Redis `status:{apiId}` .
- **Interview Qs:** naive-approach failure mode on restart; exactly-once vs. at-least-once semantics here and why at-least-once is acceptable for a health check but not for, say, a payment; retry/backoff rationale.
- **DoD:** integration test: kill the worker process mid-workflow (or simulate via Temporal test env time-skip), restart it, prove no check is lost and no duplicate `Incident` is created.

M4 – Incident engine (state machine) + flapping guard

Branch: feat/incident-engine

- [] Pure function `evaluateStreak(recentResults, N, M)` → `'OPEN' | 'RESOLVE' | 'NOOP'` , unit-tested in complete isolation from Temporal/Mongo (this is the piece to point at in interviews).
- [] Wire into the `pingApi` activity's post-processing (or a dedicated activity) to create/update/resolve `Incident` documents.

- [] Timeline sub-document is append-only; add a repository method `appendTimelineEvent` — no direct `$set` on `timeline`.
- **DoD:** table-driven unit tests covering: single blip (no incident), N-in-a-row (opens), open → partial recovery → fail again (stays open, does not reopen/duplicate), open → M-in-a-row (auto-resolves).

M5 — Notifications

Branch: `feat/notifications`

- [] Email via Resend (or Nodemailer+SMTP) on incident open/resolve; template includes API name, start time, link to incident.
- [] Fire via a lightweight queue (BullMQ on the same Redis instance) so a slow/failed email provider never blocks incident creation; failed sends retry with backoff, max 5 attempts, then logged as `notification_failed` (not silently dropped).
- [] In-app notification record (simple collection, polled by frontend for v1; websocket is a stretch goal, not v1).
- **DoD:** test proves incident creation succeeds and returns `201` even when the email provider is mocked to throw.

M6 — LangGraph AI root-cause analysis

Branch: `feat/ai-incident-analysis`

- [] LangGraph graph per §6.6, implemented as its own module (`modules/ai-analysis`), triggered async after incident creation (does not block the incident-creation response).
- [] Structured system prompt enforcing "ground in provided data only, say so if insufficient" — write this prompt to a versioned file (`prompts/root-cause-v1.md`) so future

changes are diffable and reviewable, not buried in a string literal.

- [] Timeout (10s) + circuit breaker: on Claude API failure/timeout, persist `aiSummary: null`, enqueue one retry, do not fail the parent flow.
- **DoD:** unit test with a mocked Claude client proves (a) empty failure-history short-circuits before calling Claude, (b) a timeout does not throw out of the incident-creation path.

M7 — Dashboard & frontend

Branch: `feat/dashboard`

- [] Next.js pages: API list, API detail (uptime %, latency chart via Recharts, incident feed), add-API form (client validated against the shared Zod schema from `packages/shared-types`).
- [] Data fetching via SWR with a sane revalidation interval (e.g., 15s) for the "current status" widgets — explicitly chosen over websockets for v1 simplicity.
- [] Uptime % reads from `UptimeRollup`, not raw checks (per §6.4).
- **DoD:** Lighthouse performance score ≥ 85 on the dashboard page in CI (see §10 pipeline).

M8 — Public status page + alert rules UI + hardening

Branch: `feat/public-status-and-alerts`

- [] Public, unauthenticated `GET /public/status/:teamSlug/:apiSlug`, rate-limited via Redis (`INCR + EXPIRE`, e.g. 60 req/min/IP).
- [] Explicit allowlist of fields serialized to the public response — never `res.json(apiDocument)` directly; use a dedicated `toPublicDTO()` mapper.

- [] Alert-rules UI: who gets notified per API (subset of team members).
- [] Loading/empty/error states across the app.
- **DoD:** negative security test: attempt to fetch a non-public `Api`'s status via the public route and assert `404`, not a data leak.

M9 – Deployment, docs, demo readiness

Branch: `chore/deployment-and-docs`

- [] Deploy backend + worker to Render/Railway (two separate services from one repo, or two Dockerfiles), frontend to Vercel, Mongo Atlas, Upstash Redis – all wired via CI/CD (§10).
- [] Finalize README: architecture diagram, local setup, `docker-compose up` instructions, env var table, demo GIF/Loom link.
- [] Seed script creating a demo team + an intentionally flapping demo endpoint so reviewers can watch an incident open live.
- [] Tag `v0.1.0` release.
- **DoD:** a clean `git clone` → documented steps → running local stack in under 10 minutes, verified by someone other than the author if possible.

10. CI/CD – Step-by-Step Execution Plan

Principle: every push to a feature branch runs lint/typecheck/unit tests. Every PR to `main` additionally runs integration tests against ephemeral service containers. Every merge to `main` deploys automatically. No manual `git push` to a server, ever, after M9 is complete.

10.1 Branching & merge policy

- `main` is always deployable. Protected: require PR, require CI green, require 1 approval (or self-review checklist if solo), no direct pushes.
- Feature branches: `feat/...`, `chore/...`, `fix/...` matching the milestone names in §9.
- Squash-merge to `main` to keep history readable; the PR description (not the squash commit) carries the detailed narrative.

10.2 GitHub Actions — pipeline stages

`.github/workflows/ci.yml` — runs on every push and PR:

```
name: CI
on:
  push:
    branches-ignore: [main]
  pull_request:
    branches: [main]

jobs:
  lint-and-typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck

  unit-tests:
    runs-on: ubuntu-latest
    needs: lint-and-typecheck
    steps:
```

```

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
  with:
    node-version: 20
    cache: npm
- run: npm ci
- run: npm run test:unit -- --coverage
- uses: actions/upload-artifact@v4
  with:
    name: coverage-report
    path: apps/backend/coverage

integration-tests:
  runs-on: ubuntu-latest
  needs: unit-tests
  if: github.event_name == 'pull_request'
  services:
    mongodb:
      image: mongo:7
      ports: ["27017:27017"]
    redis:
      image: redis:7
      ports: ["6379:6379"]
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
        cache: npm
    - run: npm ci
    - name: Start Temporal dev server
      run: npx @temporalio/nyc-test-env || docker
run -d -p 7233:7233 temporalio/admin-tools
    - run: npm run test:integration
      env:
        MONGO_URI:
mongodb://localhost:27017/pulse_test
        REDIS_URL: redis://localhost:6379

build:
  runs-on: ubuntu-latest
  needs: [unit-tests]

```

```

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: 20
      cache: npm
  - run: npm ci
  - run: npm run build -w apps/backend
  - run: npm run build -w apps/frontend

lighthouse:
  runs-on: ubuntu-latest
  needs: build
  if: github.event_name == 'pull_request'
  steps:
    - uses: actions/checkout@v4
    - uses: treosh/lighthouse-ci-action@v11
      with:
        urls: |
          http://localhost:3000/dashboard
        uploadArtifacts: true

```

`.github/workflows/deploy.yml` — runs only on merge to `main`:

```

name: Deploy
on:
  push:
    branches: [main]

jobs:
  deploy-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Trigger Render deploy hook (backend
        API)
        run: curl -X POST "${{
secrets.RENDER_DEPLOY_HOOK_API }}"
      - name: Trigger Render deploy hook (temporal
        worker)

```

```

    run: curl -X POST "${{
secrets.RENDER_DEPLOY_HOOK_WORKER }}"

deploy-frontend:
  runs-on: ubuntu-latest
  needs: deploy-backend
  steps:
    - uses: actions/checkout@v4
    - uses: amondnet/vercel-action@v25
    with:
      vercel-token: ${ secrets.VERCEL_TOKEN }
      vercel-org-id: ${ secrets.VERCEL_ORG_ID }
      vercel-project-id: ${
secrets.VERCEL_PROJECT_ID }
      vercel-args: "--prod"

smoke-test:
  runs-on: ubuntu-latest
  needs: [deploy-backend, deploy-frontend]
  steps:
    - name: Hit /healthz and /readyz
      run: |
        curl -f
        https://api.pulse.example.com/healthz
        curl -f
        https://api.pulse.example.com/readyz

tag-release:
  runs-on: ubuntu-latest
  needs: smoke-test
  steps:
    - uses: actions/checkout@v4
    - name: Auto-tag on successful deploy
      run: |
        git tag "deploy-$(date +%Y%m%d-%H%M%S)"
        git push origin --tags

```

Required repo secrets: `RENDER_DEPLOY_HOOK_API` ,
`RENDER_DEPLOY_HOOK_WORKER` , `VERCEL_TOKEN` , `VERCEL_ORG_ID` ,
`VERCEL_PROJECT_ID` , plus runtime secrets injected directly into

Render/Vercel/Atlas/Upstash dashboards (never into GitHub) —
MONGO_URI , REDIS_URL , JWT_SECRET , ANTHROPIC_API_KEY ,
RESEND_API_KEY , ENCRYPTION_KEY .

10.3 Jenkins alternative (if self-hosting / on-prem requirement)

Jenkinsfile at repo root, mirroring the same stages as declarative pipeline:

```
pipeline {
  agent any
  tools { nodejs "node20" }

  environment {
    MONGO_URI = credentials('pulse-mongo-uri-test')
    REDIS_URL = credentials('pulse-redis-url-test')
  }

  stages {
    stage('Install') {
      steps { sh 'npm ci' }
    }
    stage('Lint & Typecheck') {
      steps {
        sh 'npm run lint'
        sh 'npm run typecheck'
      }
    }
    stage('Unit Tests') {
      steps { sh 'npm run test:unit -- --coverage' }
      post {
        always {
          junit
          'apps/backend/coverage/junit.xml'
        }
      }
    }
  }
}
```

```

    }
    stage('Integration Tests') {
        when { changeRequest() }
        steps {
            sh 'docker compose -f docker-
compose.ci.yml up -d mongo redis temporal'
            sh 'npm run test:integration'
        }
        post {
            always { sh 'docker compose -f
docker-compose.ci.yml down -v' }
        }
    }
    stage('Build') {
        steps {
            sh 'npm run build -w apps/backend'
            sh 'npm run build -w apps/frontend'
        }
    }
    stage('Deploy') {
        when { branch 'main' }
        steps {
            sh 'curl -X POST
$RENDER_DEPLOY_HOOK_API'
            sh 'curl -X POST
$RENDER_DEPLOY_HOOK_WORKER'
            sh 'npx vercel deploy --prod --
token=$VERCEL_TOKEN'
        }
    }
    stage('Smoke Test') {
        when { branch 'main' }
        steps {
            sh 'curl -f
https://api.pulse.example.com/healthz'
            sh 'curl -f
https://api.pulse.example.com/readyz'
        }
    }
}

post {

```

```

        failure {
            echo 'Pipeline failed – check logs
above.'
        }
    }
}

```

Jenkins credentials (`pulse-mongo-uri-test` , `RENDER_DEPLOY_HOOK_API` , `VERCEL_TOKEN` , etc.) are configured in **Manage Jenkins → Credentials**, never hardcoded in the `Jenkinsfile` .

10.4 "Small change → auto-pushed" workflow (what you asked for explicitly)

1. You (or the AI agent) make a small change on a feature branch.
2. `git push origin feat/xyz` → GitHub Actions `ci.yml` runs lint/typecheck/unit tests automatically – no manual trigger needed.
3. Open a PR → `integration-tests` + `lighthouse` jobs additionally run.
4. Merge to `main` (after CI is green) → `deploy.yml` fires automatically: backend + worker redeploy on Render, frontend deploys on Vercel, a smoke test hits `/healthz` / `/readyz` , and a deploy tag is pushed.
5. Nothing is ever deployed manually once M9 is complete – `git push /merge` is the only action required.

10.5 Branch protection checklist (GitHub repo settings)

- ☐ Require status checks to pass before merging: `lint-and-typecheck` , `unit-tests` , `build` (and `integration-tests` / `lighthouse` for PRs).
- ☐ Require branches to be up to date before merging.

- [] Require linear history (squash-merge only).
- [] Restrict force-pushes and deletions on `main`.

11. Testing Strategy

Layer	Tool	Scope
Unit	Vitest/Jest	Pure functions: incident state machine, JWT utils, URL/SSRF validator, prompt-context builder
Integration	Jest + <code>mongodb-memory-server</code> + Temporal test env + <code>ioredis-mock</code> or real containers	Full request → DB round trips, workflow durability (kill/restart), notification failure isolation
API contract	Supertest	Every route in §7.2, including negative/multi-tenant-leak cases
E2E (stretch)	Playwright	Signup → register API → simulate failure → see incident on dashboard
Security	Manual + automated negative tests	SSRF guard, public-status data-leak guard, rate-limit enforcement

12. Observability & Runbook (v1-minimal but real)

- Structured JSON logging (`pino`) with a `requestId` propagated through Express middleware and into Temporal activity logs.
 - `/healthz` (process is up) and `/readyz` (Mongo + Redis reachable) on the backend; Temporal worker exposes its own metrics endpoint (default Temporal SDK Prometheus metrics).
 - On-call runbook stub in `docs/runbook.md` : what to check first if `healthCheckWorkflow` s stop progressing (Temporal Web UI → workflow list → look for stuck/failed workflows).
-

13. Interview Prep (retained & sharpened)

For each milestone, be ready to whiteboard the relevant piece of §6 (Architecture) from memory, not just describe it. The strongest signal an interviewer looks for is: *can you justify the boring infrastructure decision (Temporal, Redis cache, TTL index, repository-layer isolation) with a concrete failure mode it prevents* — not can you list the tech stack. Section 9's per-milestone "Interview Qs" are your prep checklist; answer each one in your own words before moving to the next milestone, and keep the PR description for that milestone as your written answer key.

14. What was deliberately left out of v1 (and why)

- Multi-region checks — v1 optimizes for correctness of the state machine and durability story, not geographic

redundancy; this is the natural "what would you do next" answer in an interview.

- Escalation policies beyond a flat notify-list — real on-call rotation logic (PagerDuty-style) is a product in itself; v1 proves the incident lifecycle works before layering routing complexity on top.
- Websocket-based real-time dashboard — SWR polling is simpler and sufficient at v1 scale; websockets are the correct next step once concurrent viewers per incident becomes common.